

2021-2022 Final Exam

Answer Key

CPT104 Operating Systems

Answer Key with English & Chinese Explanations

Question 1: Fundamentals (45 marks)

(a) [12 marks]

English Answer

(1) Responsiveness: Multi-threaded applications remain responsive even when part of the application is blocked. One thread can handle the user interface while another performs background computation.

(2) Resource Sharing: Threads share the same address space, code, data, and open files. This allows efficient communication between threads without requiring complex inter-process communication (IPC) mechanisms.

(3) Economy: Thread creation and context switching are significantly cheaper than those of processes. There is no memory allocation duplication since threads share the process's resources.

Additional benefits include Scalability (parallelism on multi-core processors) and overall Performance improvement.

中文解释

(1) 响应性：多线程应用在部分阻塞时仍能响应用户交互，一个线程处理UI，另一个线程后台计算。

(2) 资源共享：线程共享地址空间、代码段、数据段和打开的文件，通信效率高，无需复杂IPC机制。

(3) 经济性：线程创建和上下文切换比进程开销小得多，无需重复分配内存。

此外还包括可扩展性（多核并行）和整体性能提升。

(b) [5 marks]

English Answer

A Translation Lookaside Buffer (TLB) is a small, fast hardware cache that stores recently used page table entries. When the CPU needs to access memory, it checks the TLB first. On a TLB hit, the physical address is obtained directly without accessing the page table in main memory. This significantly reduces the effective memory access time.

中文解释

TLB是高速缓存，存放最近使用的页表项。CPU访问内存时先查TLB，命中时无需访问主存中的页表，直接得到物理地址，大幅减少内存访问时间。

(c) [9 marks]

English Answer

(1) Prevent unauthorized access between processes - ensures process isolation and data security. Each process must only access its own allocated memory regions.

(2) Protect the OS kernel from user processes - prevents user programs from corrupting kernel data structures and system tables.

(3) Control hardware access - prevents user programs from directly accessing memory-mapped I/O registers, which could compromise system stability and security.

中文解释

(1) 防止进程间未授权访问，保证进程隔离和数据安全。

(2) 保护操作系统内核不被用户进程破坏，防止篡改内核数据结构。

(3) 防止用户程序直接访问硬件I/O寄存器，保证系统稳定性。

(d) [9 marks]

English Answer

(1) Naming: Allows meaningful hierarchical names. Files in different directories can share the same name without conflict.

(2) Grouping: Enables organizing related files together, making them easier to find and manage.

(3) Efficient Searching: Tree structure reduces the search space compared to scanning all files in a flat system.

(4) Access Control: Permissions can be set at the directory level, simplifying security management.

中文解释

(1) 命名：允许分层命名，不同目录下的文件可以同名而不冲突。

(2) 分组：将相关文件组织在一起，便于查找和管理。

(3) 高效搜索：树形目录结构缩小搜索范围，比扁平结构更高效。

(4) 访问控制：可在目录级别设置权限，简化安全管理。

(e) [6 marks]

English Answer

If a process terminates, ALL its threads also terminate. Threads exist within the process context and share its resources. When the process is destroyed, all threads and their resources are destroyed together.

If all threads terminate, the process also terminates. A process is merely a container; without executing threads, there is nothing running, so the process ceases to exist.

中文解释

进程终止时所有线程也会终止，因为线程存在于进程上下文中，共享其资源。进程销毁时所有线程一并销毁。所有线程终止时进程也终止，因为没有执行单元的进程无法运行。

(f) [4 marks]

English Answer

DMA (Direct Memory Access) allows data transfer between I/O devices and memory without CPU intervention for each data unit. The DMA controller handles the entire transfer autonomously, freeing the CPU for other computation tasks. This significantly reduces CPU overhead, especially for large data transfers such as disk I/O.

中文解释

DMA允许I/O设备与内存直接传输数据，无需CPU逐个单元干预。DMA控制器自主处理整个传输过程，CPU可同时执行其他计算任务，大幅减少CPU开销，尤其适合大数据量传输。

Question 2: CPU / Memory / Disk Scheduling (31 marks)

(a) [7 marks]

English Answer

Given: P1(t=20, p=100), P2(t=30, p=145), P3(t=68, p=150)

Utilization: $U = 20/100 + 30/145 + 68/150 = 0.2000 + 0.2069 + 0.4533 = 0.8602$

Rate Monotonic assigns higher priority to shorter period:

P1 (period=100, highest priority)

P2 (period=145)

P3 (period=150, lowest priority)

RM Schedulability threshold for n=3: $3 * (2^{1/3}) - 1 = 0.7798$

Since $U = 0.8602 > 0.7798$, the task set is NOT guaranteed to be schedulable under Rate Monotonic scheduling.

中文解释

$U = 20/100 + 30/145 + 68/150 = 0.8602$

RM可调度上界 (n=3) : $3 \times (2^{1/3}) - 1 = 0.7798$

$U=0.8602 > 0.7798$, 理论上不保证可调度。

优先级: P1 (周期最短=100, 最高) > P2 (145) > P3 (150, 最低)。

(b) [6 marks]

English Answer

Reference string: 1 2 3 2 6 3 4 1 5 6 1 6 4 2, Frames = 3

Step-by-step LRU trace:

Ref=1: [1] Fault
 Ref=2: [1,2] Fault
 Ref=3: [1,2,3] Fault
 Ref=2: [1,2,3] Hit (2 recently used)
 Ref=6: [6,2,3] Fault (replace 1, least recently used)
 Ref=3: [6,2,3] Hit (3 recently used)
 Ref=4: [4,2,3] Fault (replace 6, least recently used)
 Ref=1: [4,1,3] Fault (replace 2)
 Ref=5: [5,1,3] Fault (replace 4)
 Ref=6: [5,1,6] Fault (replace 3)
 Ref=1: [5,1,6] Hit (1 recently used)
 Ref=6: [5,1,6] Hit (6 recently used)
 Ref=4: [4,1,6] Fault (replace 5)
 Ref=2: [4,2,6] Fault (replace 1)

Total page faults = 10

中文解释

使用LRU算法，3个帧，逐步追踪如上。

LRU每次替换最近最少使用的页面。

共10次缺页。

(c) [6 marks]

English Answer

Requests: 55, 58, 39, 18, 90, 160, 150, 38, 184

Disk size = 200 cylinders, Initial head position = 100

SCAN (elevator algorithm), moving toward higher cylinders first:

Order: 100 -> 150 -> 160 -> 184 -> 199 -> 90 -> 58 -> 55 -> 39 -> 38 -> 18

Movement calculation:

100->150: 50 | 150->160: 10 | 160->184: 24 | 184->199: 15

199->90: 109 | 90->58: 32 | 58->55: 3 | 55->39: 16

39->38: 1 | 38->18: 20

Total movement = 50+10+24+15+109+32+3+16+1+20 = 280 cylinders

中文解释

SCAN算法先向高柱面方向扫描到最大请求184（再到端点199），然后反转方向。

扫描顺序：100->150->160->184->199->90->58->55->39->38->18

总移动量 = 280 柱面。

(d) [5 marks]

English Answer

Exponential averaging formula: $\tau(n+1) = \alpha * T(n) + (1 - \alpha) * \tau(n)$

Given: $\tau_1 = 11$, $\alpha = 0.3$

$$\begin{aligned}\tau_2 &= 0.3 * 5 + 0.7 * 11 = 1.5 + 7.7 = 9.2 \\ \tau_3 &= 0.3 * 2 + 0.7 * 9.2 = 0.6 + 6.44 = 7.04 \\ \tau_4 &= 0.3 * 6 + 0.7 * 7.04 = 1.8 + 4.928 = 6.728 \\ \tau_5 &= 0.3 * 4 + 0.7 * 6.728 = 1.2 + 4.7096 = 5.9096\end{aligned}$$

Prediction for the 5th process burst: approximately 5.91 ms

中文解释

指数平均公式: $\tau(n+1) = \alpha \times T(n) + (1-\alpha) \times \tau(n)$

$\tau_1=11$, $\alpha=0.3$

$\tau_2=9.2$, $\tau_3=7.04$, $\tau_4=6.728$, $\tau_5=5.9096$

第5个进程的预测CPU突发时间约为5.91ms。

(e) [7 marks]

English Answer

Given: P1=67000, P2=65000, P3=60000 bytes, each in a partition of 70000 bytes.

Internal fragmentation (waste inside each partition):

P1: 70000 - 67000 = 3000 bytes

P2: 70000 - 65000 = 5000 bytes

P3: 70000 - 60000 = 10000 bytes

Total internal fragmentation = 18000 bytes

Can a new process of 15000 bytes be accommodated? NO.

Internal fragmentation waste is trapped inside already-allocated partitions and cannot be reclaimed. The largest free contiguous block available = 0 bytes.

中文解释

内部碎片（每个分区内的浪费）：

P1: 3000字节, P2: 5000字节, P3: 10000字节

总内部碎片 = 18000字节

15000字节的新进程无法容纳。内部碎片被锁在已分配分区中无法回收，当前没有可用的连续空闲空间。

Question 3: Resource Allocation (Banker's Algorithm) (12 marks)

English Answer

Given resources: Total R1=6, R2=4, R3=4, R4=2

Need matrix (Max - Allocation):

P1: (1, 2, 0, 0) P2: (0, 1, 0, 2) P3: (0, 0, 2, 0)

P4: (2, 1, 0, 0) P5: (2, 1, 0, 0)

Available = (1, 1, 2, 0)

Safety check:

Step 1: P3 needs (0,0,2,0) $\leq$ (1,1,2,0) YES $\rightarrow$ Available = (2, 2, 2, 0)

Step 2: P1 needs (1,2,0,0) $\leq$ (2,2,2,0) YES $\rightarrow$ Available = (4, 2, 3, 1)

Step 3: P4 needs (2,1,0,0) $\leq$ (4,2,3,1) YES $\rightarrow$ Available = (5, 3, 4, 1)

Step 4: P5 needs (2,1,0,0) $\leq$ (5,3,4,1) YES $\rightarrow$ Available = (5, 3, 4, 2)

Step 5: P2 needs (0,1,0,2) $\leq$ (5,3,4,2) YES $\rightarrow$ Available = (6, 4, 4, 4)

Safe sequence: P3 $\rightarrow$ P1 $\rightarrow$ P4 $\rightarrow$ P5 $\rightarrow$ P2

Conclusion: The system is in a SAFE state.

中文解释

Need矩阵 = Max - Allocation。

Available = (1,1,2,0)

安全检查步骤:

P3需要(0,0,2,0) $\leq$ (1,1,2,0) $\rightarrow$ Available=(2,2,2,0)

P1需要(1,2,0,0) $\leq$ (2,2,2,0) $\rightarrow$ Available=(4,2,3,1)

P4需要(2,1,0,0) $\leq$ (4,2,3,1) $\rightarrow$ Available=(5,3,4,1)

P5需要(2,1,0,0) $\leq$ (5,3,4,1) $\rightarrow$ Available=(5,3,4,2)

P2需要(0,1,0,2) $\leq$ (5,3,4,2) $\rightarrow$ 全部完成

安全序列: P3 $\rightarrow$ P1 $\rightarrow$ P4 $\rightarrow$ P5 $\rightarrow$ P2

系统处于安全状态。

Question 4: OS in C (Readers-Writers Problem) (12 marks)

(a) [4 marks]

English Answer

Limitation: Writer starvation.

If readers continuously arrive, the `rw_mutex` is always held by at least one reader. A writer waiting to write may be blocked indefinitely because new readers keep entering the critical section. This is a fairness problem - the writer can never acquire `rw_mutex`.

中文解释

写者饥饿问题。

如果读者不断到达, `rw_mutex`始终被至少一个读者持有。等待写入的写者可能被无限期阻塞, 因为新读者不断进入临界区。这是一个公平性问题。

(b) [4 marks]

English Answer

Removing mutex from the reader code causes a race condition on `readcount`.

Two readers could simultaneously read the old value of `readcount` (e.g., both see `readcount=0`), both increment to 1, and both call `wait(rw_mutex)`. This decrements `rw_mutex` twice but it was only incremented once (or zero times),

leading to deadlock. The semaphore becomes inconsistent.

中文解释

移除读者代码中的mutex会导致readcount上的竞态条件。

两个读者可能同时读到readcount的旧值（如都读到0），都递增为1，都调用wait(rw_mutex)。rw_mutex被减两次但只加了一次，导致信号量不一致，可能引发死锁。

(c) [4 marks]

English Answer

If only rw_mutex is used in the reader (removing the mutex and readcount logic), only ONE reader can access the resource at a time.

This violates the fundamental requirement of the readers-writers problem, which states that multiple readers should be allowed to read simultaneously. The solution essentially degrades to simple mutual exclusion.

中文解释

如果读者只使用rw_mutex（移除mutex和readcount逻辑），一次只能有一个读者访问资源。

这违反了读者-写者问题的基本要求，即允许多个读者同时读取。该方案退化成了简单的互斥访问。